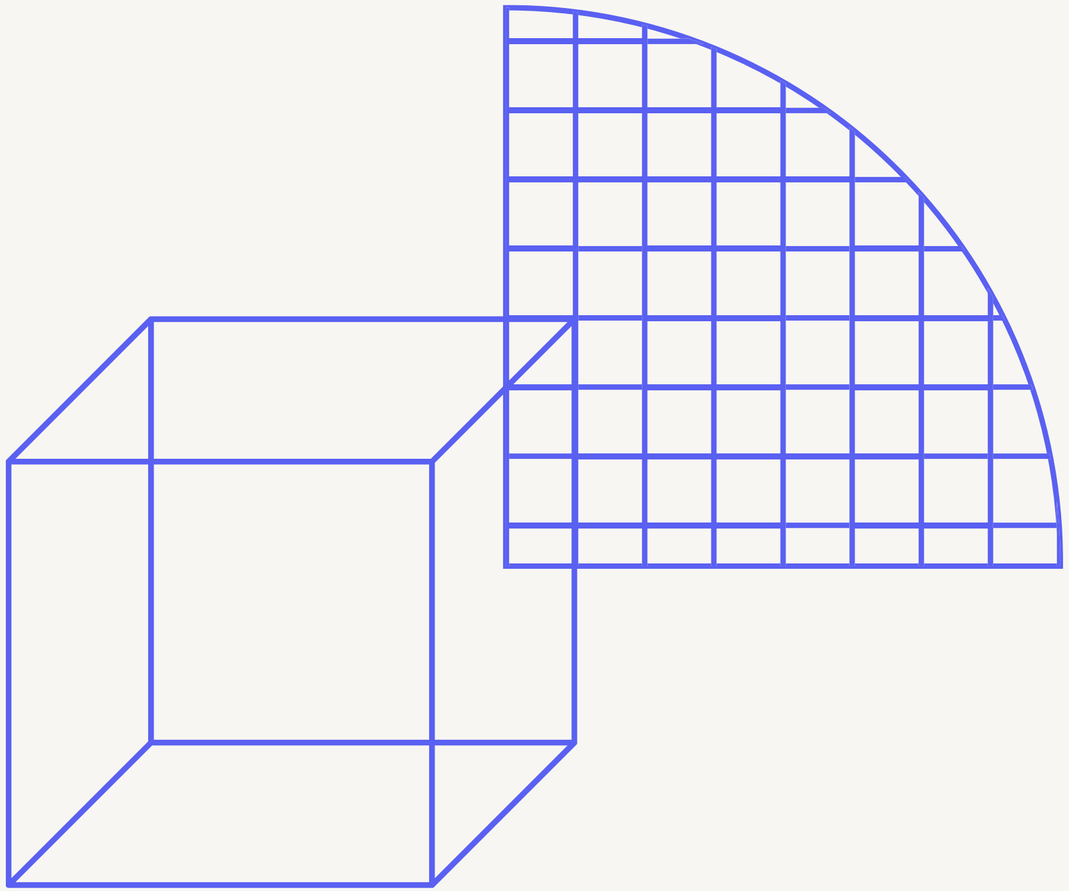
ΓΛΩΣΣΕΣ SERVER-SIDE ΔΙΑΔΙΚΤΥΑΚΟΥ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ (PHP)

ΔΙΑΛΕΞΗ 3



ΤΣΙΑΜΗΣ ΘΕΟΔΩΡΟΣ

Κεφάλαιο 3ο

Αντικειμενοστραφής PHP

- Εισαγωγή
- Ιδιότητες και Μέθοδοι.
- Static Ιδιότητες/Μέθοδοι.
- Σταθερές.
- Magic Methods
- Contructors και Destructors.
- Getteres και Setters.
- __call()
- __callStatic()
- Serialization
- __sleep()
- __wakeup()
- Cloning
- Callable Classes

Εισαγωγή

- **Στην PHP όπως και σε άλλες αντικειμενοστραφείς γλώσσες:**
  - Ορίζουμε κλάσεις με τη λέξη κλειδί **class.**
  - Δημιουργούμε νέα instances με τη λέξη κλειδί **new.**
  - Εισάγουμε κληρονομικότητα με τη λέξη κλειδί **extends.**
  - Έλεγχος πρόσβασης με τα **public, protected, private.**
  - **try...catch** όπως και στη Java (Διαθέτει όμως πολύ λιγότερα built-in extensions).
  - Abstract classes με τη λέξη κλειδί **abstract.**

Ιδιότητες και Μέθοδοι

- Στην PHP ο τελεστής πρόσβασης σε ιδιότητες και μεθόδους είναι ο **->** (σε αντίθεση με την τελεία . της Java και της JavaScript).
- **$this:** Αντίστοιχο του **this** της Java.

Ιδιότητες και Μέθοδοι - Παράδειγμα

```
class BankAccount{
    private $balance;
    public function setBalance($balance){
        $this->balance = $balance;
    }
    public function getBalance(){
        return $this->balance;
    }
}
```

Static Ιδιότητες/Μέθοδοι

- Όπως και στη Java χρησιμοποιούμε τη λέξη κλειδί **static** για να ορίσουμε static μεταβλητές.
- Για να έχουμε πρόσβαση σε static ιδιότητες/μεθόδους, δεν μπορούμε να χρησιμοποιήσουμε τον τελεστή **->**, αλλά χρησιμοποιείται ο **::**
- Η λέξη κλειδί **self** ισοδυναμεί με τη χρήση του ονόματος της κλάσης (συνιστάται η χρήση της για λόγους ευκολότερης συντήρησης).

Static Ιδιότητες/Μέθοδοι - Παράδειγμα

```
class BankAccount {
    private static $bankname;
    public function getBankName(){
        return self::bankname;
    }
}
```

Σταθερές

- Χρησιμοποιείται η λέξη κλειδί **const.**
- Προσπέλαση σε αυτές γίνεται με τον ίδιο τρόπο με τις staic ιδιότητες.
- Το όνομά τους ΔΕΝ αρχίζει με $.
- Όπως και στη Java, για μεθόδους διατίθεται η λέξη κλειδί **final.**

Σταθερές - Παράδειγμα

```
class BankAccount {
    const MIN_BALANCE = 20;
    public __construct($initial_balance){
        if($initial_balance < self::MIN_BALANCE) {
            throw new Exception('More money needed :(');
        }
    }
}
```

Magic Methods

- Η PHP εξ’ ορισμού ορίζει ένα πλήθος μεθόδων που ξεκινούν με **__ (δύο κάτω παύλες)** οι οποίες όταν οριστούν σε μία κλάση επιτελούν κάποια ειδική λειτουργία.
- Ενέργειες για τις οποίες στη Java κάνουμε override μεθόδους της τάξης Object, στην PHP υλοποιούνται με τις “μαγικές μεθόδους”
- **Π.χ. __toString, __clone**

Constructors and Destructors

- **Κατασκευαστές (Constructors)** ορίζουμε με την “μαγική μέθοδο” **__construct.**
  - Μόνο ένας κατασκευαστής επιτρέπεται σε κάθε ορισμό αντικειμένου.
  - Σε περίπτωση που επιθυμούμε πολυμορφισμό κατασκευαστών, μόνη λύση είναι ο έλεγχος των παραμέτρων όπως στη JavaScript.

- **Destructors** αντίστοιχα με την **__destruct.**

- Παράδειγμα:

```
function __construct(){
    $this->balance = 50;
}
```

Getters and Setters

- Τι γίνεται όταν επιχειρείται προσπέλαση ή τροποποίηση ανύπαρκτης ή κρυφής ιδιότητας;
- Μπορούμε να το ορίσουμε αν δηλώσουμε τις “μαγικές μεθόδους” **__get($property)** και **__set($property, $value).**
- Με κατάλληλους ελέγχους μπορούμε να ορίσουμε διαφορετική συμπεριφορά ανάλογα με το όνομα (ή ακόμη και την τιμή σε περίπτωση set).
- Συνήθως χρησιμοποιείται για να έχουμε ιδιότητες οι οποίες φαίνονται σαν public αλλά όταν διαβάζονται ή τροποποιούνται εκτελείται και επιπλέον κώδικας.
- Μας επιτρέπουν να αποφύγουμε τη δήλωση περιττών getters και setters για προληπτικούς λόγους, όπως μας αναγκάζει και η Java.

Η “μαγική μέθοδος” __call()

- Αν έχει οριστεί, καλείται όταν επιχειρηθεί να κληθεί μη ορατή μέθοδος (είτε λόγω πρόσβασης, είτε επειδή δεν υπάρχει).
- Αφορά κλήση σε **instance.**
- Καλείται με πρώτη παράμετρο το όνομα της μεθόδου και δεύτερη ένα array με τις παραμέτρους.

```
public function __call($name, $arguments) {
  echo "Calling object method '$name' " . implode(', ', $arguments);
}
```

Η “μαγική μέθοδος” __callStatic()

- Αν έχει οριστεί, καλείται όταν επιχειρηθεί να κληθεί μη ορατή μέθοδος (είτε λόγω πρόσβασης, είτε επειδή δεν υπάρχει).
- Αφορά κλήση σε **στατικό context.**
- Καλείται με πρώτη παράμετρο το όνομα της μεθόδου και δεύτερη ένα array με τις παραμέτρους.

```
public static function __callStatic($name, $arguments) {
  echo "Calling static method '$name' " . implode(', ', $arguments);
}
```

Serialization

- Η PHP διαθέτει τις μεθόδους **serialize()** και **unserialize()** για τη μετατροπή ενός αντικειμένου σε string ή την αποκωδικοποίηση μίας τέτοιας συμβολοσειράς αντίστοιχα.
- Συχνά χρησιμοποιούνται για αποθήκευση αντικειμένων σε βάση δεδομένων, αν και συνήθως κάτι τέτοιο δυσχεραίνει τον χειρισμό των δεδομένων.

Η “μαγική μέθοδος” __sleep()

- Όταν καλείται η **serialize()** για κάποιο αντικείμενο, καλείται πρώτα η μέθοδος **__sleep()** αν έχει οριστεί σε αυτό.
- Χρησιμοποιείται για να γίνει κάποιος πρώτος καθαρισμός του αντικειμένου πριν την σειριοποίηση, καθώς πρέπει αν επιστρέφει ένα array με τα ονόματα των ιδιοτήτων που πρέπει να σειριοποιηθούν.

Η “μαγική μέθοδος” __wakeup()

- Αντίστοιχα με προηγουμένως, η **__wakeup()** καλείται όταν κληθεί η **unserialize().**
- Μπορεί να χρησιμοποιηθεί για να ανακτήσει πράγματα που χάθηκαν κατά την σειριοποίηση (π.χ. κάποια σύνδεση με μία βάση δεδομένων).

Cloning

- Κλωνοποιούμε ένα αντικείμενο χρησιμοποιώντας τη λέξη κλειδί **clone.**
- **Shallow copy** των ιδιοτήτων.
- Αν έχει οριστεί η μέθοδος **__clone()** στο αντικείμενο, καλείται μετά τη δημιουργία του κλώνου για να αλλάξει τυχόν μεταβλητές που απαιτείται (π.χ. για deep copying).
- Παράδειγμα σύνταξης:

```
$copy_of_object = clone $object;
```

Callable classes

- Από την PHP 5.3 υποστηρίζεται η “μαγική μέθοδος” **__invoke**, η οποία καλείται όταν επιχειρηθεί να κληθεί ένα στιγμιότυπο ως συνάρτηση.
- Ελέγχουμε αν ένα αντικείμενο είναι callable με τη συνάρτηση **is_callable().**

Callable classes - Παράδειγμα

```
class CallableClass {
    function __invoke($x) {
        var_dump($x);
    }
}
$obj = new CallableClass();
$obj(5); // int(5)
var_dump(is_callable($obj)); // bool(true)
```

Κεφάλαιο 4ο

Built-in Objects and Functions

- Συναρτήσεις για χειρισμό String.
- Συναρτήσεις για χειρισμό Arrays.
- Έξοδος.
- Μαθηματικές συναρτήσεις.

Συναρτήσεις για χειρισμό String

- **~100 built-in συναρτήσεις.**
- **Οι πιο συχνές:**
  - **strlen():** μήκος συμβολοσειράς σε χαρακτήρες.
  - **strtolower(), strtoupper(), ucfirst(), ucwords():** μετατροπές case.
  - **trim(), ltrim(), rtrim():** αφαίρεση κενών από την αρχή και/ή το τέλος.
  - **explode():** διάσπαση string σε τμήματα και επιστροφή τους σε array.
  - **substr():** επιστροφή τμήματος μίας συμβολοσειράς.
  - **strpos(), stripos():** η θέση στην οποία υπάρχει (αν υπάρχει) μία συμβολοσειρά.
  - **unlencode(), urldecode():** (απο)κωδικοποίηση σευμβολοσειράς βάσει του τρόπου που χρησιμοποιείται στα URL για GET parameters.

Συναρτήσεις για χειρισμό Arrays

- **~80 built-in συναρτήσεις.**
- **Οι πιο συχνές:**
  - **sizeof():** πλήθος τιμών στον πίνακα.
  - **implode():** ένωση τιμών του πίνακα σε string (αντίστροφη της explode()).
  - **in_array():** έλεγχος παρουσίας τιμής στον πίνακα.
  - **array_keys():** επιστροφή των κλειδιών σε νέο πίνακα.
  - **array_merge():** συγχώνευση πινάκων.
  - **array_unique():** εξάλειψη διπλότυπων.
  - **asort(), arsort(), ksort(), krsort(), usort(), uasort(), uksort():** ταξινόμηση του πίνακα.

Έξοδος

- Είτε για εκτύπωση της διεπαφής και των δεδομένων, είτε για debugging.
- Οι πιο συνηθισμένες είναι οι **echo** και **print**, οι οποίες τυπώνουν μία συμβολοσειρά στην έξοδο.

- Άλλοι τρόποι για την έξοδο δεδομένων:
  - **number_format(), money_format()**
  - **printf(), sprintf()**

- Άλλοι τρόποι για debugging:
  - **print_r():** για εκτύπωση πινάκων αναδρομικά.
  - **var_dump():** εκτύπωση τύπου και τιμής.

Μαθηματικές συναρτήσεις

- **~50 built-in συναρτήσεις.**
- **Οι πιο συχνές:**
  - **abs():** απόλυτη τιμή.
  - **round(), ceil(), floor():** στρογγυλοποιήσεις.
  - **is_finite(), is_infinite(), is_nan():** έλεγχος του τύπου του αριθμού.
  - **rand(), srand():** δημιουργία ψευδοτυχαίων αριθμών.

ΤΕΛΟΣ 3ης ΔΙΑΛΕΞΗΣ

Ευχαριστώ για την προσοχή σας!